МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования

«Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

ОТЧЕТ

по лабораторной работе №6

по дисциплине

«Технологии компонентного программирования»

«Потоковые web-серверы и голосовой запрос к серверу»

Выполнил:
Лебедевич Артем Владимирович
магистрант группы 556301

Преподаватель:
Герман Олег Витольдович
кандидат технических наук
доцент кафедры ИТАС

Минск, 2026 г.

СОДЕРЖАНИЕ

1 Цель работы 2

2 Теоретические сведения 3
2.1 Потоковый web-сервер 3
2.2 Признаки MFCC 3
2.3 Отличия от методических материалов 4

3 Реализация 5
3.1 Структура программы 5
3.2 Схема взаимодействия 5
3.3 Запись и прослушивание WAV 5
3.4 Классификатор 6
3.5 Сервер 7
3.6 Клиент 7

4 Результаты работы программы 9

5 Ответы на контрольные вопросы 12

Заключение 13

Список использованных источников 14

1 ЦЕЛЬ РАБОТЫ

Изучить потоковую передачу изображений и звука web-сервером на Python, запись и воспроизведение WAV-файлов, преобразование текста в речь и речи в текст, выделение признаков MFCC и классификацию WAV-файлов. Используя примеры методических материалов, создать приложение, в котором клиент записывает голосовой запрос, распознаёт его и передаёт текст серверу, а сервер преобразует текст в WAV, классификатором определяет, что запрошено, и возвращает ответ в виде WAV-файла, который воспроизводится на стороне клиента.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Потоковый web-сервер

Модуль `http.server` стандартной библиотеки содержит класс `BaseHTTPRequestHandler`: методы `do_GET` и `do_POST` записывают в поток `wfile` строку состояния, заголовки и тело ответа. Потоковым ответ становится, когда тело передаётся частями по мере чтения файла или генерации данных, а не формируется целиком в памяти. Основные понятия приведены в таблице 2.1.

Таблица 2.1 – Понятия потоковой передачи

Понятие	Смысл
`Content-Type`	тип тела: `image/png`, `audio/wav`, `application/json`
`multipart/x-mixed-replace`	сервер присылает изображения одно за другим, и каждое следующее заменяет предыдущее в теге `<img>`
`Range: bytes=a-b` и ответ `206`	запрос части файла; проигрыватель `<audio>` использует его для перемотки, а WebKit (Safari, встроенный браузер окна) без него не воспроизводит звук
WAV	заголовок RIFF (44 байта) и отсчёты PCM; здесь 16 бит, моно, 16 кГц
TTS	синтез речи из текста: `say` в macOS, `pyttsx3` (SAPI, eSpeak) в других системах
Распознавание речи	`speech_recognition`: метод `recognize_google` отправляет звук в бесплатный сервис Google Web Speech
MFCC	мел-кепстральные коэффициенты: БПФ окон по 2048 отсчётов, банк мел-фильтров, логарифм и ДКП — 13 чисел на окно
SVM	метод опорных векторов, класс `SVC` библиотеки scikit-learn

2.2 Признаки MFCC

Шкала мел отражает нелинейность слуха: человек хорошо различает 100 и 200 Гц, но не различает 15 000 и 15 100 Гц. Сигнал делится на перекрывающиеся окна, спектр каждого окна пропускается через банк треугольных фильтров, расставленных равномерно по шкале мел, энергии фильтров логарифмируются, а дискретное косинусное преобразование декоррелирует их. Первые 13 коэффициентов описывают огибающую спектра — то, какой звук произносится, почти независимо от высоты голоса.

2.3 Отличия от методических материалов

– Для записи и воспроизведения вместо PyAudio используется `sounddevice`: он обращается к той же библиотеке PortAudio, но устанавливается из готового пакета без сборки. Воспроизведение, как в программе методички, идёт блоками по 1024 отсчёта.
– `pyttsx3` в macOS не сохраняет WAV, поэтому речь синтезируется командой `say --data-format=LEI16@16000`; в Windows и Linux используется `pyttsx3`.
– `adjust_for_ambient_noise` по умолчанию поглощает первую секунду записи, а короткий запрос длится около секунды; длительность уменьшена до 0,2 с.
– Классификатор методички (среднее MFCC и линейный SVC) на пяти файлах проверялся на обучающих же данных. Здесь обучающий набор озвучен восемью голосами, точность измеряется на голосах и фразах, не участвовавших в обучении, и признаки дополнены (раздел 3.4).

3 РЕАЛИЗАЦИЯ

3.1 Структура программы

Листинг 3.1 – Структура лабораторной работы

```
lab6/voice.py       запись, воспроизведение, текст -> WAV, WAV -> текст
lab6/classifier.py  обучающий набор TTS, признаки MFCC, классификатор SVM
lab6/server.py      потоковый сервер http.server (порт 8050) и ответы
lab6/page.html      страница: вопрос, запись WAV в браузере, плеер, картинки
lab6/client.py      клиент: запись -> распознавание -> вопрос -> ответ
lab6/cases.py       проверяемые сценарии
lab6/runner.py, gui.py  консоль и окно wxPython
```

3.2 Схема взаимодействия

```
клиент                                  сервер (lab6/server.py)
микрофон -> recording.wav (sounddevice)
recording.wav -> текст (Google Web Speech)
текст --- POST /ask {"text"} ----------> текст -> query.wav (TTS)
                                        query.wav -> MFCC -> SVM -> тема
                                        тема -> ответ -> answer.wav (TTS)
answer.wav <-- GET /audio/answer_*.wav --- блоками по 16 КиБ, Range
динамики <- answer.wav (блоки по 1024 отсчёта)
```

Без доступа к сервису распознавания клиент отправляет саму запись (`POST /ask-wav`), и сервер классифицирует её напрямую.

3.3 Запись и прослушивание WAV

Класс `Recorder` (листинг 3.2) открывает входной поток микрофона и собирает блоки отсчётов между вызовами `start` и `stop`; метод `stop` сохраняет WAV. Функция `play` читает файл и записывает его в выходной поток блоками по 1024 отсчёта; флаг позволяет прервать воспроизведение.

Листинг 3.2 – Запись и воспроизведение (lab6/voice.py)

```
class Recorder:
    def start(self) -> None:
        self._chunks = []
        self._stream = sd.InputStream(samplerate=self.rate, channels=1,
                                      dtype="int16", callback=self._collect)
        self._stream.start()

    def _collect(self, data, _frames, _time, _status) -> None:
        self._chunks.append(data[:, 0].copy())

    def stop(self, path: Path) -> Path:
        self._stream.stop(); self._stream.close(); self._stream = None
        return write_wav(path, np.concatenate(self._chunks), self.rate)

def play(path: Path, wait: bool = True) -> None:
    rate, samples = read_wav(path)
    with sd.OutputStream(samplerate=rate, channels=1, dtype="int16") as out:
        for start in range(0, len(samples), 1024):
            out.write(samples[start : start + 1024].reshape(-1, 1))
```

В окне программы кнопки Record и Stop записывают файл в каталог `artifacts/lab6/client/`, кнопка Play (или двойной щелчок) воспроизводит выбранную запись, Ask the server отправляет её на распознавание и серверу, Upload загружает её на сервер. Страница сервера записывает звук и в браузере: `getUserMedia` даёт поток микрофона, отсчёты кодируются в 16-битный WAV на JavaScript и отправляются запросом `POST /upload`; все записи выводятся с проигрывателем `<audio>`.

3.4 Классификатор

Обучающий набор строится синтезом речи: шесть тем (приветствие, время, дата, шутка, музыка, картинка), по шесть формулировок, каждая произнесена восемью голосами `say` — 288 WAV-файлов. Признаки одного файла (листинг 3.3): после обрезки тишины вычисляются MFCC (13 коэффициентов в окне 2048 отсчётов с шагом 512, как в методичке), затем среднее, разброс и среднее изменение коэффициентов по всей фразе и среднее по каждой трети фразы — всего 78 чисел. Трети сохраняют порядок звуков, который теряется при усреднении.

Листинг 3.3 – Признаки WAV-файла (lab6/classifier.py)

```
def mfcc(path: Path) -> np.ndarray:
    audio, rate = librosa.load(str(path), sr=16000)
    trimmed, _bounds = librosa.effects.trim(audio, top_db=30)
    return librosa.feature.mfcc(y=trimmed, sr=rate, n_mfcc=13,
                                n_fft=2048, hop_length=512).T

def features(path: Path) -> np.ndarray:
    frames = mfcc(path)
    delta = np.diff(frames, axis=0)
    thirds = [part.mean(axis=0) for part in np.array_split(frames, 3)]
    return np.concatenate([frames.mean(axis=0), frames.std(axis=0),
                           np.abs(delta).mean(axis=0), *thirds])
```

Модель — `StandardScaler` и `SVC` с ядром RBF. Точность проверяется перекрёстной проверкой с группировкой: в одном случае тестовые голоса не участвуют в обучении, в другом — тестовые формулировки (таблица 3.1). Обученная модель сохраняется в `artifacts/lab6/model.pkl` и переобучается при изменении тем или голосов.

Таблица 3.1 – Точность классификации (6 тем, случайный выбор — 17 %)

Модель	Новый голос	Новая формулировка
Методичка: среднее MFCC, линейный SVC	43 %	40 %
78 признаков, линейный SVC	55 %	43 %
78 признаков, SVC с ядром RBF	60 %	53 %

Сервер всегда озвучивает текст одним голосом, поэтому различаются

только формулировки: фразы обучающего набора распознаются все, а для новой формулировки тема определяется примерно в половине случаев.

3.5 Сервер

Сервер (листинг 3.4) отвечает на запросы, перечисленные в таблице 3.2. Классификатор загружается при первом вопросе. Ответ на тему «время» и «дата» вычисляется, на тему «шутка» выбирается из списка, на тему «музыка» к речи добавляется мелодия из синусоидальных нот, на тему «картинка» возвращается ещё и ссылка на изображение.

Таблица 3.2 – Запросы к серверу

Запрос	Ответ
`GET /`	страница с вопросом, записью, проигрывателем и картинками
`GET /image?text=`	PNG, нарисованный Pillow (пример `/image` методички)
`GET /stream-image`	кадры JPEG часов в `multipart/x-mixed-replace`, раз в секунду
`GET /audio/<файл>`, `/stream/<файл>`	WAV блоками по 16 КиБ с поддержкой `Range`
`GET /recordings`	список загруженных записей в JSON
`POST /upload?name=`	сохранение записи клиента
`POST /ask`	текст → WAV → тема → текст ответа и WAV ответа
`POST /ask-wav`	то же для записи без текста

Листинг 3.4 – Потоковая передача WAV (lab6/server.py, фрагмент)

```
self.send_response(206 if found else 200)
self.send_header("Content-Type", "audio/wav")
self.send_header("Accept-Ranges", "bytes")
self.send_header("Content-Length", str(end - start + 1))
if found:
    self.send_header("Content-Range", f"bytes {start}-{end}/{size}")
self.end_headers()
with file.open("rb") as source:
    source.seek(start)
    left = end - start + 1
    while left > 0:
        chunk = source.read(min(CHUNK, left))
        self.wfile.write(chunk)
        left -= len(chunk)
```

3.6 Клиент

Метод `converse` класса `VoiceClient` (листинг 3.5) распознаёт запись, отправляет текст и загружает WAV ответа блоками. Консольный клиент `python -m lab6.client` записывает 4 с с микрофона, проигрывает запись, задаёт вопрос и проигрывает ответ.

Листинг 3.5 – Диалог с сервером (lab6/client.py)

```
def converse(self, wav: Path) -> tuple[dict, Path]:
    try:
        reply = self.ask_text(voice.recognize(wav))
    except voice.RecognitionError as exc:
        reply = self.ask_wav(wav)          # offline: the server classifies the WAV
        reply["recognition"] = str(exc)
    return reply, self.download(str(reply["audio"]))
```

4 РЕЗУЛЬТАТЫ РАБОТЫ ПРОГРАММЫ

На рисунке 4.1 показано окно после выполнения всех сценариев (9 из 9): запись с вопросом «show me a picture» распознана, сервер определил тему «picture», ответ воспроизведён и показана картинка. Для снимка запись озвучена голосом Daniel, а не записана с микрофона.

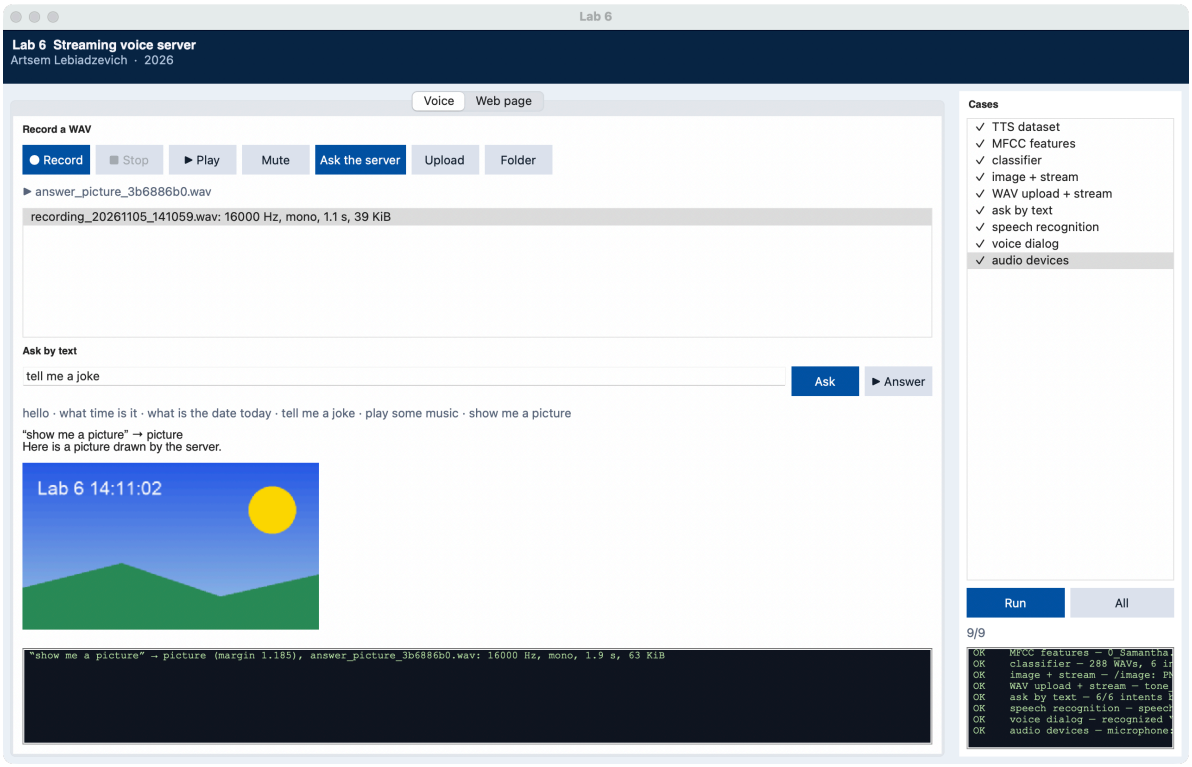


Рисунок 4.1 – Окно лабораторной работы: запись, ответ сервера и список проверок

На рисунке 4.2 — страница сервера во встроенном браузере: ответ на вопрос «play some music» с проигрывателем, запись WAV, поток кадров часов и изображение.

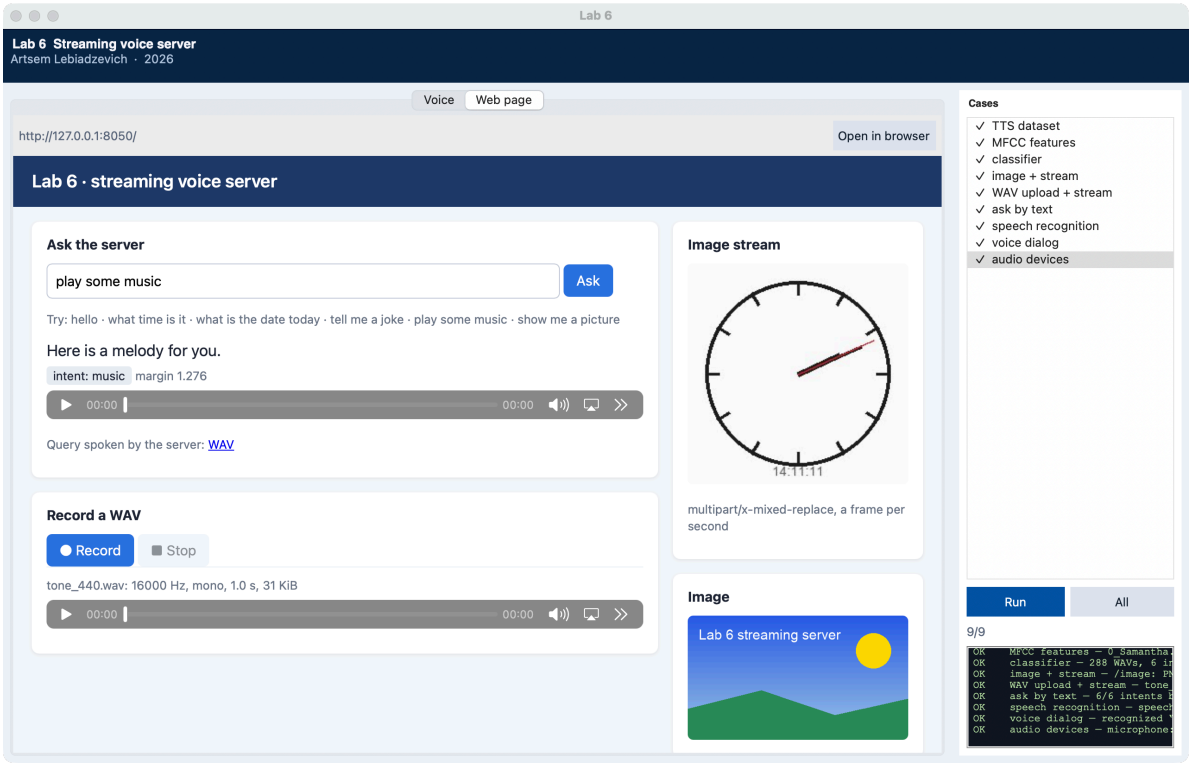


Рисунок 4.2 – Страница потокового сервера

Команда `python main.py --lab 6` выполняет сценарии без окна; вывод приведён в листинге 4.1, содержание проверок — в таблице 4.1.

Листинг 4.1 – Вывод программы

```
tts_dataset -> 288 WAVs = 6 intents x 6 phrasings x 8 voices, 0.4-2.1 s
mfcc_features -> 0_Samantha.wav: MFCC 17 frames x 13, feature vector of 78
classifier_accuracy -> 288 WAVs, 6 intents, 8 voices; accuracy: train 100%, unseen voice 60%, unseen phrasing 53%
image_endpoints -> /image: PNG 480x270; /stream-image: 3 JPEG frames in multipart/x-mixed-replace
wav_upload_and_stream -> tone_440.wav: 16000 Hz, mono, 1.0 s, 31 KiB; same bytes back; Range 0-43 -> 206 with the RIFF
      header; listed in /recordings
ask_by_text -> 6/6 intents by their first phrasing; time -> "It is 14:12." + answer WAV
speech_recognition -> speech.wav -> "tell me a joke"
voice_dialog -> recognized "play some music" -> music -> answer_music_8cf0bc8b.wav: 16000 Hz, mono, 3.8 s, 117 KiB
audio_devices -> microphone: MacBook Pro Microphone; speakers: MacBook Pro Speakers
```

Таблица 4.1 – Проверяемые сценарии

Сценарий	Что проверяется
TTS dataset	288 WAV-файлов, ни один не короче 0,2 с
MFCC features	матрица MFCC из 13 столбцов, вектор из 78 признаков
classifier	точность на обучающих данных не ниже 90 %, на новых формулировках не ниже 35 %
image + stream	`/image` отдаёт PNG; `/stream-image?frames=3` — три корректных кадра JPEG
WAV upload + stream	загруженный WAV возвращается без изменений; запрос `Range: bytes=0-43` даёт ответ 206 с заголовком RIFF; запись есть в списке
ask by text	темы определяются по первой формулировке каждой, WAV ответа длиннее 0,5 с; ответ о времени содержит текущий час
speech recognition	речь «tell me a joke» распознаётся Google (без сети сценарий пропускается)
voice dialog	полный цикл: WAV → текст → сервер → тема «music» → WAV ответа
audio devices	найдены микрофон и динамики

Автоматические тесты (`python -m unittest tests.test_labs`) повторяют эти проверки; все тесты проходят.

5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Как web-сервер на Python возвращает изображение или звук? Устанавливает код 200 и заголовок `Content-Type` (`image/png`, `audio/wav`) и записывает байты файла в `wfile`; браузер выводит их тегом `<img>` или `<audio>`.

2. Чем потоковая передача отличается от обычной? Тело ответа передаётся частями по мере чтения или создания данных. Для изображений используется `multipart/x-mixed-replace`, для звука — передача блоками и запросы `Range`.

3. Как записать и воспроизвести WAV на Python? Входной поток PortAudio (`sounddevice` или PyAudio) отдаёт блоки отсчётов, которые модуль `wave` сохраняет в файл; для воспроизведения файл читается блоками и записывается в выходной поток.

4. Что такое MFCC? Мел-кепстральные коэффициенты: спектр окна сигнала, пропущенный через банк фильтров по шкале мел, прологарифмированный и преобразованный ДКП. Они описывают огибающую спектра и служат признаками речи.

5. Как построить классификатор WAV-файлов? Подготовить размеченные файлы, вычислить для каждого вектор признаков (MFCC и их статистики), обучить классификатор (`SVC`) и проверять точность на файлах, не участвовавших в обучении.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучены потоковые web-серверы на `http.server`, запись и воспроизведение WAV-файлов, синтез и распознавание речи, признаки MFCC и классификация звука. Создано приложение по схеме методических материалов: клиент записывает голосовой запрос, распознаёт его и отправляет текст серверу; сервер озвучивает текст, классификатором определяет тему и возвращает ответ в виде WAV-файла, который воспроизводится у клиента. Сервер также передаёт изображение, поток кадров и записи клиента.

Практический вывод: классификатор из пяти файлов методички проверялся на тех же файлах и не показывал реальной точности. При честной проверке среднее MFCC узнаёт новую формулировку в 40 % случаев; признаки, сохраняющие порядок звуков, и нелинейное ядро повышают точность до 53 %, а надёжно распознаются фразы из обучающего набора. Для браузера на WebKit сервер должен поддерживать запросы `Range`, иначе звук не воспроизводится.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Герман, О. В. Потоковые web-серверы: лекция и лабораторная работа по дисциплине «Технологии компонентного программирования» / О. В. Герман. — Минск: БГУИР, 2024.
2. http.server — HTTP servers [Электронный ресурс]. — Режим доступа: https://docs.python.org/3/library/http.server.html.
3. librosa: audio and music processing in Python [Электронный ресурс]. — Режим доступа: https://librosa.org/doc/latest/.
4. scikit-learn: Support Vector Machines [Электронный ресурс]. — Режим доступа: https://scikit-learn.org/stable/modules/svm.html.
5. SpeechRecognition [Электронный ресурс]. — Режим доступа: https://pypi.org/project/SpeechRecognition/.
6. python-sounddevice [Электронный ресурс]. — Режим доступа: https://python-sounddevice.readthedocs.io/.